



Doc2Query in Practice (Required Reading - Also a Final Project Idea)

Completion requirements

Doc2Query in Practice

Boosting Retrieval Quality for Small RAG Systems

Some of you are working on problems that use or could benefit from Doc2Query—for example, building an assistant to help students, faculty, and staff quickly obtain grounded answers from the Centre College Student Handbook with citations. This short guide is meant to help you understand the method and see how to implement it in your own projects.

Doc2Query is another extension to RAG that nicely complements HyDE, which we studied earlier in the course. You should expect a question or two about this topic on the second paper test. Familiarity with Doc2Query may also prove helpful when interviewing for roles in data science or machine learning engineering.

1. Why Doc2Query?

Retrieval-augmented generation (RAG) combines a generator (an LLM) with a retriever over an external knowledge source. The original RAG paper by Lewis et al. showed that adding a dense retriever over Wikipedia significantly improved factual performance on knowledge-intensive tasks compared to using the parametric model alone [Lewis et al., 2020].

However, even a good retriever can miss relevant passages when:

- Users phrase questions differently from how documents are written (“skip class” vs. “unexcused absence”), or
- Policies are expressed in very specific language, while users ask fuzzy, informal questions.

This is often called **vocabulary mismatch**. In educational or enterprise settings (student handbooks, internal policies, FAQs), it shows up as “the answer *is* in the corpus, but the system fails to retrieve it.”

Doc2Query is a family of techniques designed to reduce this mismatch by expanding documents with **synthetic queries**—questions or search phrases the document could plausibly answer. Instead of waiting for the user to phrase things perfectly, we pre-bake likely queries and index them.

2. The Core Idea: Document-Side Query Prediction

The original **Doc2Query** method (Nogueira et al., 2019) trains a sequence-to-sequence model to predict likely user queries for a given document, using large-scale search logs and relevance labels. At index time, each document is expanded with these predicted queries before being fed into a traditional BM25 search engine [Nogueira et al., 2019]. In later work, **docTTTTTquery** (“docT5query”) swaps in T5 and achieves even better retrieval performance [Nogueira & Lin, 2019].

Conceptually:

1. Take a passage or document.
2. Generate several queries that this passage could answer.
3. Attach those queries to the document (for BM25) or store them as separate items (for dense retrieval).
4. At search time, match user queries against this expanded representation.

In modern RAG practice with LLMs and embeddings, we often **don’t** retrain a model from scratch. Instead, we:

- Use a general-purpose LLM to generate a handful of questions per chunk.
- Use an off-the-shelf embedding model to represent those questions as vectors.
- Build a K-nearest neighbor (KNN or ANN) index over question vectors, each pointing back to its source chunk.

In short, giving each chunk several different ‘ways of speaking’ makes it much more likely that at least one synthetic question will match a real user’s query.

3. Where Doc2Query Fits in a RAG Pipeline

Doc2Query lives in the **offline indexing** part of your RAG stack. You use it when you’re preparing your corpus, not while you’re answering questions in real time.

In a typical RAG system, you can think of the pipeline in five stages:

1. **Ingest and chunk.**

First, you ingest your source documents and break them into chunks that are big enough to carry context but small enough to stay focused—usually a few hundred words with titles, URLs, and stable IDs attached. This is the same chunking step you would do for a vanilla RAG system.

2. **Generate synthetic questions.**

Next, for each chunk, you use an LLM to generate a small set of questions (for example, 3–5) that a real user might ask and that this chunk can clearly answer.

These are your “pseudo-queries” or **doc2query questions**.

3. **Build a question index.**

You then treat those questions as the main objects for retrieval. Each record might look like

```
{"question": "...", "chunk_id": "...", "section_title": "...", "url": "..."}.
```

You embed all of the questions with an off-the-shelf embedding model and build a vector index (for example, a NumPy array with cosine similarity, FAISS, or a lightweight vector database). Each question vector points back to its source chunk.

4. **Serve queries.**

At runtime, when a user asks a question, you embed the user’s query and search over the **synthetic questions** in your index. You retrieve the top-k closest synthetic questions, map them back to their underlying chunks, and collect a small set of distinct passages to work with. This gives you a focused, evidence-ready context.

5. **Answer with evidence.**

Finally, you either answer directly by showing the retrieved snippet with its section title and URL, or you pass the snippets into an answer-generation prompt that tells the model to respond *only* using those chunks. If similarity scores are weak across the board, you fall back to “insufficient context” instead of guessing.

In other words, doc2query doesn’t change your generation step. It changes **what you retrieve** and **how well** your retrieval matches the way real users ask questions.

4. A Tiny Worked Example: Practical Test 2 (Tell-Tale Heart)

On **Practical Test 2** in DSC 360, you worked with a deliberately minimal doc2query-style setup built around Poe’s *The Tell-Tale Heart* (`heart.txt`). The surrounding code was already doing most of the heavy lifting:

- `index.py` – a script that:
 - Chunks the story into passages.

- Uses an LLM to generate several questions about each passage.
 - Calls a helper to **embed those synthetic questions**.
 - Saves an index: a NumPy array of vectors plus a question→chunk mapping.
- `ask.py` – a script that:
 - Embeds a user query using the same embedding helper.
 - Finds the closest synthetic questions in the index.
 - Retrieves the linked chunks and has a model answer using those chunks.
- `embed_questions()` – the small component you had to complete:
 - Wraps the embedding model call.
 - Normalizes and returns vectors in the format expected by `index.py` and `ask.py`.
 - Ideally, we would have put that in a separate file `embed.py` and imported into both.

This is **doc2query in miniature**:

- You never trained a new model; you simply **used** an LLM to generate questions and a separate model to embed them.
- The system indexed **questions only, not the original chunks**, and then mapped back from questions → chunks when answering.
- There was **no filtering** of the generated questions; every question the LLM produced was embedded and indexed, even if it wasn't great.
- Evaluation was lightweight: you could check whether the right passage showed up in the top-k for a few test questions.

For many of you, this was a first exposure to the idea that you can improve retrieval not only by choosing a different embedding model, but also by choosing **what** you embed. The Tell-Tale Heart setup was intentionally small for use on a test. For a mini-capstone like the Student Handbook project—and certainly for real-world systems—you should treat it as a starting point rather than a finished design and aim for a slightly more robust pipeline.

- Consider **filtering** synthetic questions that the chunk cannot clearly answer.
- Consider indexing **both** generated questions and the **original chunks** (e.g., a hybrid between question embeddings and chunk embeddings).
- Design a more systematic **evaluation** on a small “gold” question set, not just a couple of ad hoc queries.

The handbook project takes the same basic pattern you saw on Practical Test 2, but expects a more careful, realistic implementation with better retrieval quality and stronger safety guardrails.

5. Mini-Capstone Idea: Centre Student Handbook Q&A

As a small final project, you can build a **Doc2Query-powered RAG helper** for the Centre Student Handbook:

Goal (MVP): Given a student’s question about a policy (absences, alcohol, housing, Title IX, etc.), return a short answer supported by an accurate snippet and a citation to the relevant section. If you can’t find a clear answer, say “insufficient context.”

The steps below define a **minimum viable product** that a single student can realistically complete in about two weeks. You do **not** need to train any models or build a web UI; a clean CLI plus a small index is enough. You are encouraged to reuse patterns and code from earlier labs (semantic search, RAG, embedding helpers).

5.1 Choose scope and collect sections

Start by limiting your scope. Pick **6–8 handbook sections** that are realistic and useful, such as:

- Attendance and unexcused absences
- Alcohol and drug policy
- Housing and quiet hours
- Academic honesty
- Title IX / sexual misconduct contacts
- Campus safety and emergency procedures

Copy each selected section into a separate `.txt` file under `data/` (you may copy/paste rather than scrape). For each section, record its canonical URL so you can cite it later.

5.2 Chunk with metadata

Next, break each section into chunks that are big enough to give context but small enough to stay focused—aim for roughly **200–300 words per chunk**. Treat a chunk as a single coherent policy unit (often one paragraph or a couple of closely related paragraphs). Avoid chunks shorter than ~80–100 words by merging very short bits with their neighbors, and avoid chunks longer than ~350–400 words by splitting at sensible boundaries. Keep headings or section labels attached to the relevant chunk.

Write a simple script (`ingest.py`) that:

- Reads each `.txt` file.
- Splits it into chunks (e.g., by headings or blank lines, with a little manual tuning if needed).
- Assigns each chunk a stable `chunk_id`.

- Stores records like:

```
{
  "chunk_id": "attendance-003",
  "section_title": "Attendance Policy",
  "url": "https://www.centre.edu/...",
  "last_updated": "2024-08-01",
  "text": "Full text of this chunk..."
}
```

Save all chunk records to `index/chunks.jsonl`. This is your basic handbook corpus.

5.3 Generate doc2query questions

Now you will build your doc2query expansion.

Write `index.py` so that, for each chunk in `chunks.jsonl`:

1. You send a prompt to an LLM such as:
"You are helping students search the Centre Student Handbook. Given the passage below, write 3–5 student-style questions that this passage can clearly answer. Use natural language a student would type into a search box. Avoid questions that require information not in the passage."
2. You get back several questions and store them as records like:

```
{
  "question_id": "attendance-003-q2",
  "chunk_id": "attendance-003",
  "question": "How many unexcused absences am I allowed before I face penalties?"
}
```

Add all question records to `index/questions.jsonl`. For the MVP, you can skip fancy filtering: if a question is obviously nonsense you can discard it, but you do not need a complex classifier.

5.4 Embed and index the synthetic questions

Still in `index.py`, you will embed the questions and build a tiny vector index:

- Use a small embedding model (e.g., `all-MiniLM-L6-v2` or a local Ollama embedder) to compute vectors for each `question`.
- Build an $N \times d$ NumPy array of embeddings and save it as `index/embeddings.npy`.
- Optionally normalise vectors so cosine similarity reduces to a dot product.
- Save a small `index/meta.json` with the embedding model name, dimension, and any options.

At this point you have a **doc2query question index** for the subset of the handbook you chose.

5.5 Implement a REPL (**ask.py**)

Now you will build a minimal query interface.

Write a tiny CLI program **ask.py** that:

1. Loads **meta.json**, **questions.jsonl**, **chunks.jsonl**, and **embeddings.npy**.
2. Reads a user's question from stdin (a single line is fine).
3. Embeds the user's question and computes cosine similarity against all question vectors.
4. Picks the top-k synthetic questions (e.g., **k = 5**) and maps them to their chunks.
5. Builds a compact context containing the best 1–3 distinct chunks, plus their titles and URLs.
6. Calls a generator model with a prompt like:
"Answer the student's question using only the handbook excerpts below. If the excerpts don't clearly answer the question, say 'insufficient context'. After your answer, list the section title and URL for each excerpt you used."
7. Prints:
 - A 1–2 sentence answer, and
 - Citations such as: "Attendance Policy – Student Handbook (URL)".

If similarity scores are low across the board (for example, all below some cutoff), skip generation and respond with a simple message like: "Insufficient context; try a more specific question." This keeps the system honest and avoids obvious hallucinations.

A plain-text REPL is all that is required for this project; a web UI is optional.

5.6 Evaluate your system

Finally, you should evaluate whether your retrieval is working.

Before you write any code, design a tiny test set:

- Draft at least **20 "gold" questions** that matter to real students, covering your chosen sections.
- For each, decide which chunk(s) count as "correct" (one or a small set).

Then, after you have built the pipeline:

- Run your 20 questions through **ask.py**.

- Check whether the correct chunk appears in the **top-3** retrieved chunks for each question.
- For numeric/policy answers (e.g., “How many unexcused absences?”), verify that:
 - The retrieved snippet contains the actual number or key phrase, and
 - The generated answer uses that exact value, not a guess.

You do not need a full-blown metrics script; a simple table in your README (question, expected chunk, retrieved chunks, pass/fail and a short comment) is enough for this mini-capstone.

In your [README.md](#), summarise:

- Which questions passed and which failed.
- Your best guess at **why** failures happened (chunking, question generation, embeddings, or the answer-generation prompt).
- One concrete idea you would try next if you had more time.

5.7 Extensions for a Two-Person Team

If two people take on this project together and finish the MVP early, there are several natural extensions that deepen the work without blowing up the scope:

- **Question filtering.**
Add a simple filter that discards synthetic questions the chunk cannot clearly answer (for example, using word-overlap heuristics or a small “Can this passage answer this question? YES/NO” LLM check). Compare retrieval performance before and after filtering.
- **Hybrid indexing.**
Embed both the synthetic questions and the original chunks. At query time, retrieve from both indexes and merge results (for example, by taking the best score across both). Evaluate whether this improves recall on your 20 gold questions.
- **HyDE-style query expansion (building on hybrid indexing).**
Reuse your chunk embeddings and add a simple HyDE step: given a user question, have the LLM generate a short “hypothetical answer passage,” embed that passage once, and use its vector to retrieve similar **chunks**. Then compare:
 - doc2query-only retrieval vs.
 - doc2query + HyDE-over-chunks.
 You don’t need a new model—just one extra prompt and embedding call per query.
- **Richer evaluation.**
Script a tiny evaluation harness that computes Hit@1 and Hit@3 for your gold questions automatically, and perhaps adds a small qualitative error log for failures.

- **Lightweight UI.**

Wrap `ask.py` in a very small web interface (for example, Flask or Streamlit) so a user can type a question into a text box and see the answer plus citations in the browser. Keep the UI minimal; the focus is still on the retrieval design.

Any one of these would make a solid extension for a two-person team and would bring the project closer to what you might build in an early-career data science or ML engineering role.

6. Design Choices and Common Pitfalls

Doc2Query techniques can work very well, but there are a few traps worth watching for as you move beyond the minimal “Tell-Tale Heart” example.

6.1 Question quality and hallucination

The LLM might generate questions that the chunk **cannot actually answer**. For example, a housing policy passage that mentions quiet hours on weekdays might attract a synthetic question about “weekend quiet hours” that aren’t specified. If those hallucinated questions get indexed, they can pull the retriever toward irrelevant or ambiguous chunks.

Doc2Query-- (Gospodinov et al., 2023) shows that **filtering out poor-quality synthetic queries can simultaneously improve retrieval quality and reduce index size**, by removing noisy expansions that don’t help [Gospodinov et al., 2023].

In a small project, you can approximate this filtering with simple heuristics:

- Automatically discard questions that contain names, numbers, or entities not present in the chunk text.
- Require at least one meaningful word overlap (beyond stopwords) between the question and the chunk.
- Optionally, ask a second LLM: “Can this passage clearly answer the question above? Answer YES or NO.” and drop questions that get a “NO.”

Even lightweight checks like these can prevent the worst mismatches without making the project unmanageable.

6.2 How many questions per chunk?

More questions per chunk give you **more ways to match** user queries, but they also produce **larger indexes and more computation**.

For a handbook-scale project, 3–5 questions per chunk is usually plenty. If you have time, you can experiment with 1, 3, and 5 questions per chunk and see how Hit@3 changes on your gold test set. You may find that you get most of the benefit from the first few questions and only a small gain from adding more.

6.3 Chunking and overlap

If chunks are too long, they can exceed the model’s context or drag in unrelated text. If they are too short, they lack enough context for an embedding model or an LLM to make sense of them.

For the handbook project, practical guidelines are:

- Aim for a “Goldilocks zone” of about **150–300 words per chunk**.
- Use small overlaps at paragraph or sentence boundaries so you don’t cut policies in awkward places.
- Keep headings or section labels attached; they often contain the keywords students will use in their questions.

Good chunking makes doc2query more effective, because the synthetic questions will be more coherent and focused, and you’ll retrieve policy units that are easy to quote.

6.4 When doc2query is overkill

You probably **don’t** need doc2query if:

- The corpus is tiny (e.g., 10 short FAQs); you can just brute-force search.
- Exact keyword search or BM25 already works extremely well.
- The content changes constantly and you can’t afford to regenerate synthetic questions.

On the other hand, doc2query shines when:

- The corpus is moderately large and fairly stable (like a yearly student handbook).
- Users ask in their own words, and you want your system to “meet them where they are.”
- You care about explainability: doc2query systems make it easy to show “here is the passage we used.”

7. Related Ideas: HyDE and Multi-Query Retrieval

Doc2Query is **document-side expansion**: you enrich documents or chunks *before* search. You generate queries from documents, index those, and then match user questions against this expanded view of the corpus.

The **HyDE** approach (Gao et al., 2023) is more about **query-side expansion**. Given a user query, HyDE:

1. Uses an LLM to generate a short *hypothetical document* that might answer the question.
2. Embeds this fake document once.
3. Uses that embedding to retrieve similar *real* documents from the index.

This can be powerful when you cannot preprocess the corpus (for example, web-scale search or rapidly changing content), or when the user’s query is very short or vague. In our course, you saw HyDE mainly as “invent a plausible answer, then use its embedding as a better query” on Practical Test 1.

Simple **multi-query** retrieval is another query-side trick: you generate several paraphrases of the user’s question, embed them all, and merge results (e.g., by taking the best score across paraphrases). This gives the query multiple “angles” without changing the documents at all.

In practice, these techniques can be combined with doc2query—for example, a HyDE-style hypothetical document searching over a doc2query-expanded index, or multi-query retrieval plus document-side expansion. For your student projects, though, it is best to get a clean doc2query pipeline working first and only then experiment with HyDE or multi-query as optional extensions.

8. Conclusion

Doc2Query is one concrete way of taking control of retrieval instead of treating it as a black box. Rather than asking “Which embedding model should I use?” it pushes you to ask more interesting questions:

- How should we break the corpus into chunks that make sense to a reader?
- What kinds of questions do we expect people to ask?
- How can we expand or filter those questions so retrieval is precise and honest?
- What simple tests will tell us whether the system is actually finding the right passages?

In this handout, doc2query sits next to ideas like HyDE and multi-query retrieval as part of a small toolbox for improving RAG. All of them share the same theme: you can reshape either the documents, the queries, or both, and then measure the effect.

If you can explain how you would apply these ideas to a new corpus—how you would choose chunks, generate and filter questions, and evaluate results—you are already thinking at the level expected of someone building real retrieval systems, whether in a research project, a capstone, or an early-career data science or ML engineering role.

References

- Gao, L., Ma, X., Lin, J., & Callan, J. (2023). *Precise Zero-Shot Dense Retrieval without Relevance Labels* (HyDE). In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023).
<https://aclanthology.org/2023.acl-long.99/>
- Gospodinov, M., MacAvaney, S., & Macdonald, C. (2023). *Doc2Query--: When Less is More*. In *Advances in Information Retrieval: 45th European Conference on Information Retrieval (ECIR 2023)*. Preprint: <https://arxiv.org/abs/2301.03266>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
<https://arxiv.org/abs/2005.11401>
- Nogueira, R., Yang, W., Lin, J., & Cho, K. (2019). *Document Expansion by Query Prediction*. arXiv:1904.08375. <https://arxiv.org/abs/1904.08375>
- Nogueira, R., & Lin, J. (2019). *From doc2query to docTTTTTquery*. Micropublication on MS MARCO passage retrieval. PDF:
https://cs.uwaterloo.ca/~jimmylin/publications/Nogueira_Lin_2019_docTTTTTquery-v2.pdf
- nyu-dl. (2019). *Doc2query: Document Expansion by Query Prediction* (code repository). GitHub. <https://github.com/nyu-dl/dl4ir-doc2query>
- Castorini. (2019). *docTTTTTquery* (code repository and pretrained expansions). GitHub. <https://github.com/castorini/docTTTTTquery>
- Soyuj. (2023, June 20). *Document Expansion: Overcoming Vocabulary Mismatch with Doc2Query*. Blog post. <https://soyuj.com/blog/document-expansion/>

Last modified: Thursday, November 13, 2025, 11:51 AM